

Chapter 3. REGISTERS

This Week

Important operations

Registers with Parallel-Load

Registers with Clear

Registers with Increment

Next Week

Combining Load and Clear inputs

Shift (and Serial-Load)

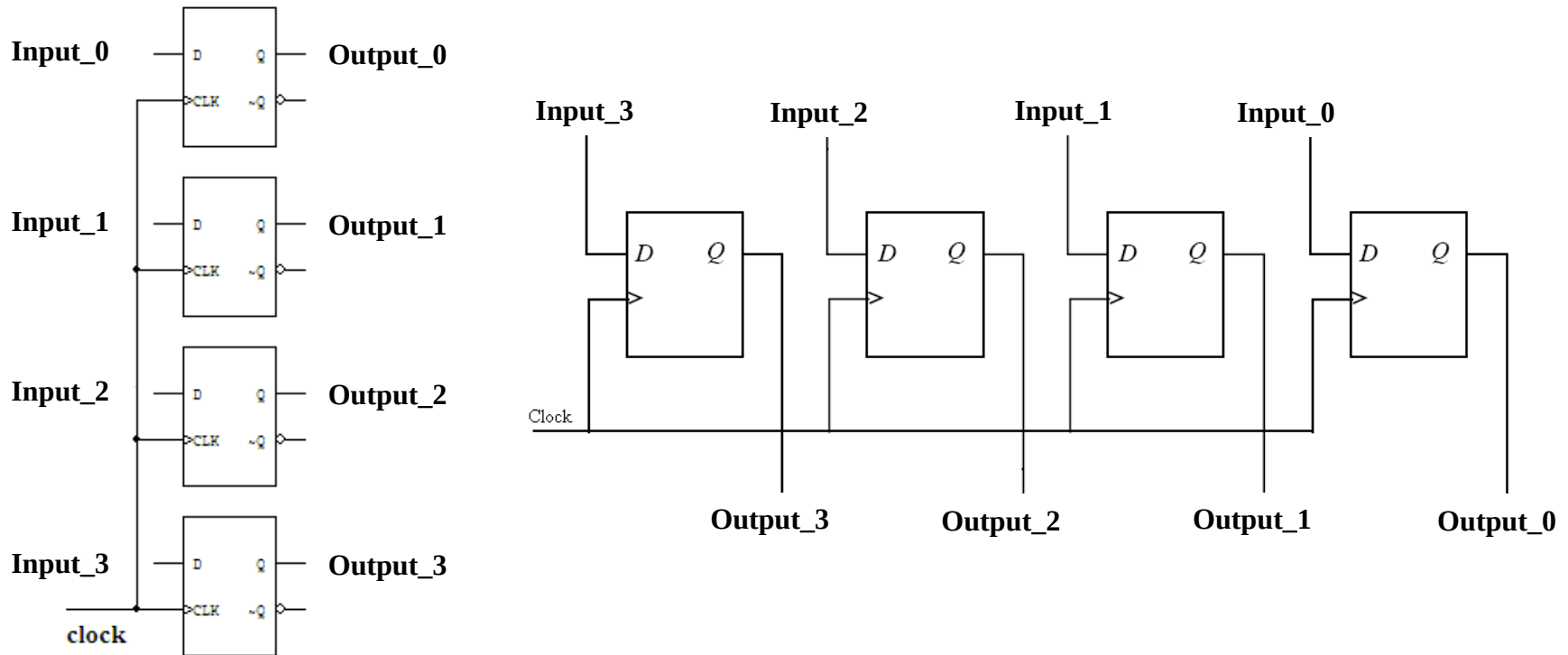
Combining Parallel-Load and Shift inputs

Universal Register Design

Registers

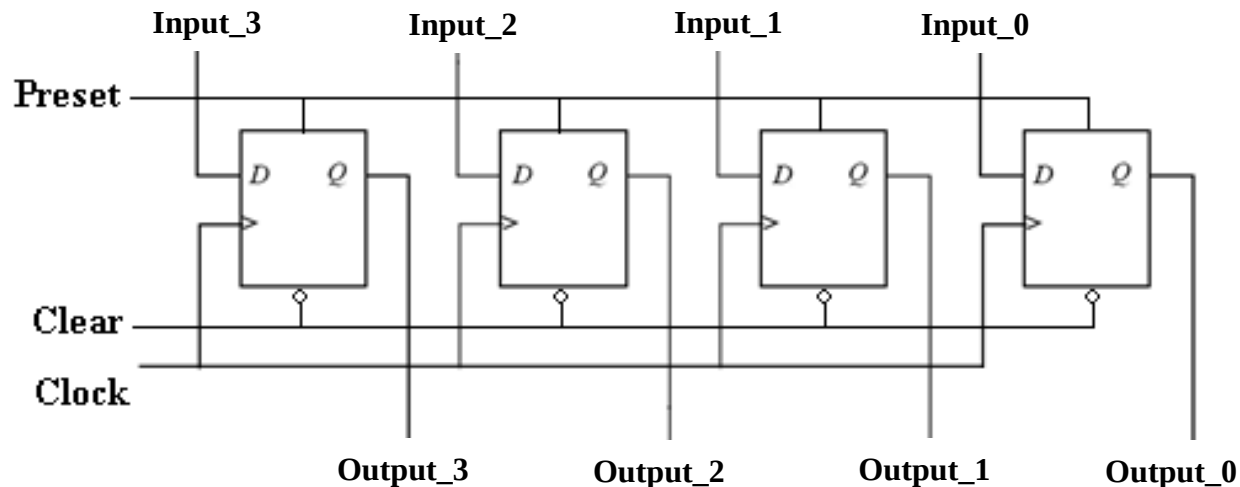
Registers are memory components made by bringing multiple flip-flops together. A register with n flip-flops in it, has a capacity to store n bits of data.

Example: The 4-bit register seen below is made with D-type flip-flops.



Preset and Clear Inputs

Registers have asynchronous *Preset* and *Clear* inputs to set the initial value. These inputs are called “asynchronous” because they operate independently of the clock signal.



Registers with important operations

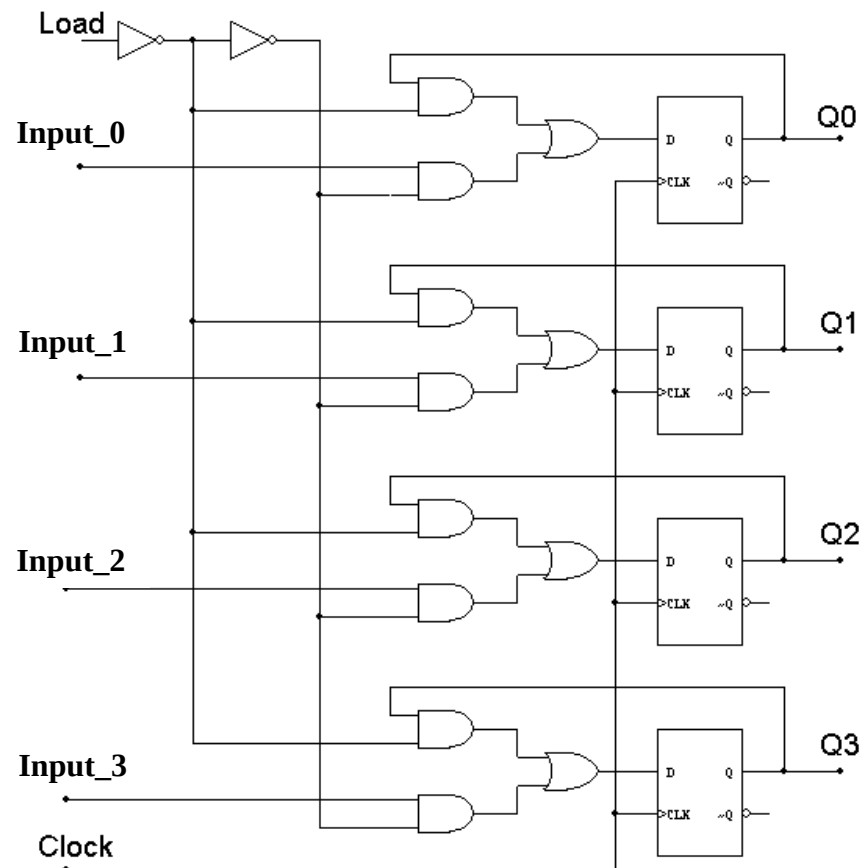
- Registers with parallel load
- Registers with clear
- Registers with set
- Shift registers and loading serial data
- Increment and decrement registers
- Registers with two's complement
- Transferring value of a register to another register

The operations above can be combined to obtain a register with multiple operations.

Registers with Parallel Load

Design a 4-bit register using D-type flip-flops, which;

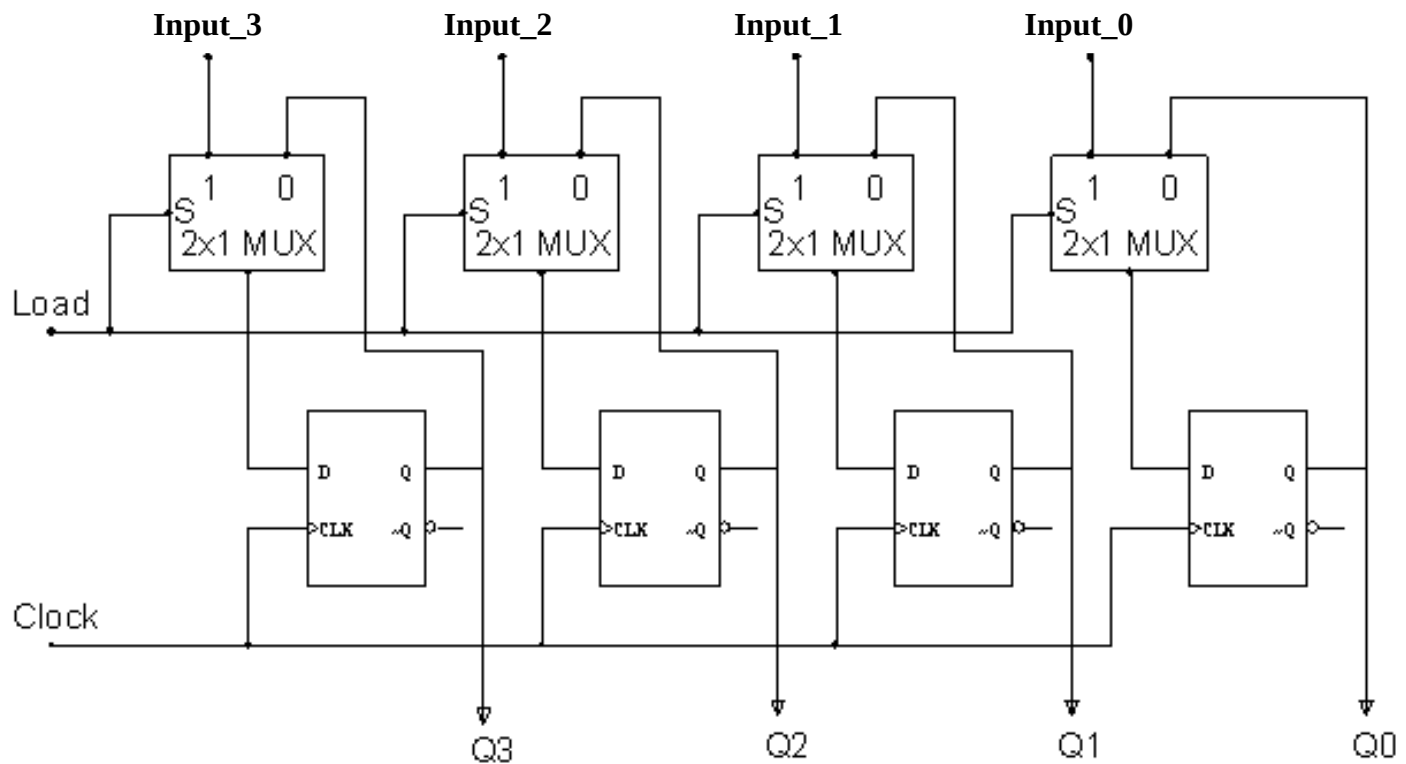
- Keeps the output value when **Load** = 0
- Loads the Input data when **Load** = 1



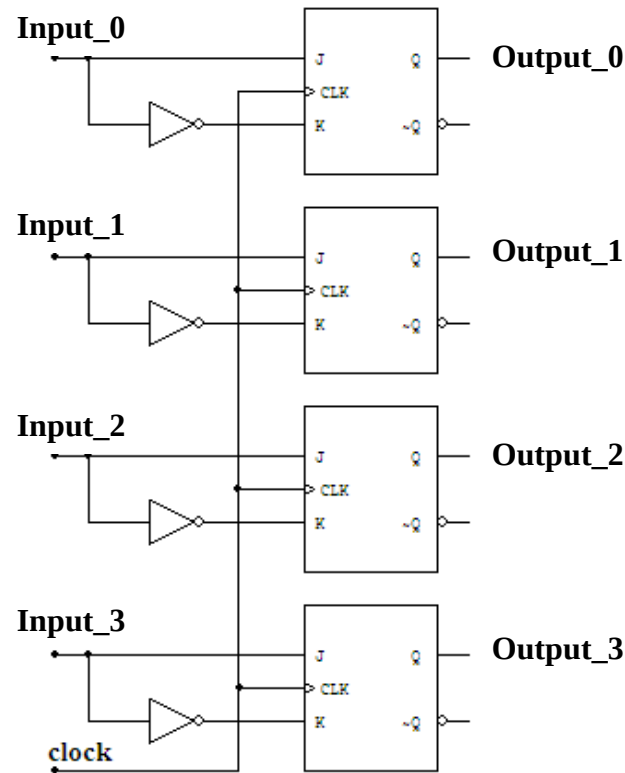
Registers with Parallel Load

We can also design a register with parallel load using D-type flip flops by using 2x1 multiplexers.

Characteristic equation for a 2x1 MUX is; $y = S'.I_0 + S.I_1$



Registers with Set and Clear (JK-Type Flip-Flops)



To *Reset*, we input 0's, and to *Set*, we input 1's.

Registers with Parallel Load (JK-Type Flip-Flops)

To design a register with *Load* input using JK-FF, we can use the state table of a 1-bit register, given below.

Control Input	Data Input	Next State	Inputs of Flip-Flops	
Load	Input_0	Q	J	K
0	0	Hold the state	0	0
0	1	Hold the state	0	0
1	0	Data input (0)	0	1
1	1	Data input (1)	1	0

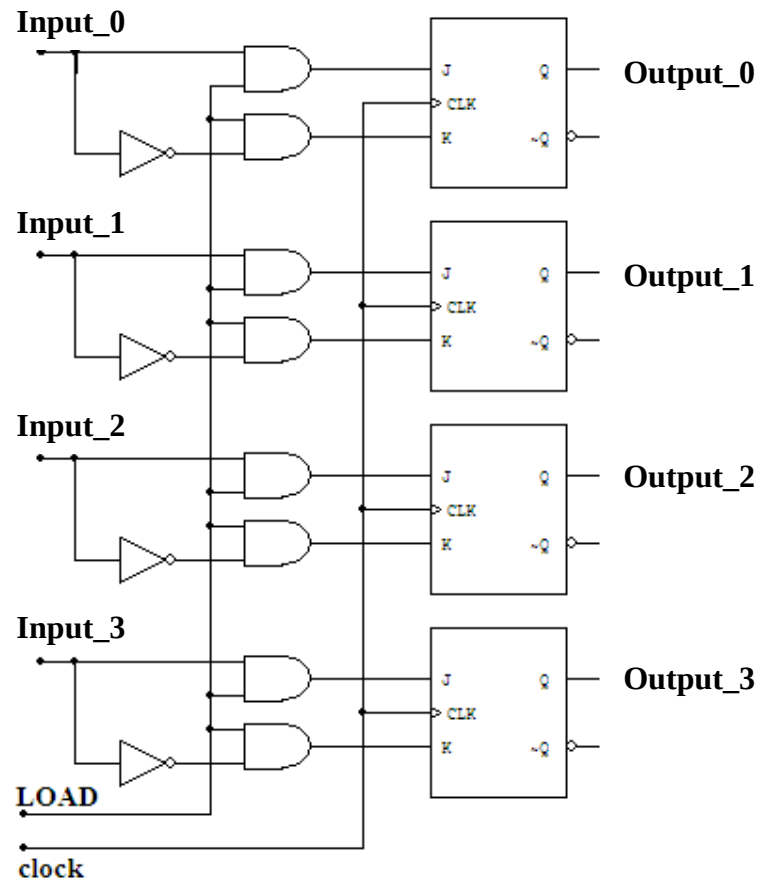
$$J = \text{Load} \cdot \text{Input_0}$$

$$K = \text{Load} \cdot \text{Input_0}'$$

Registers with Parallel Load (JK-Type Flip-Flops)

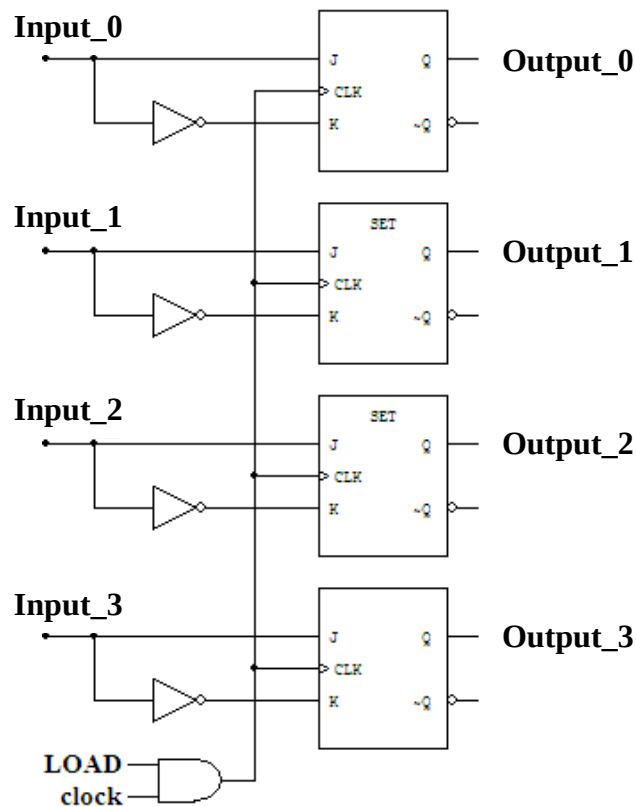
$J = \text{Load} \cdot \text{Input}_0$

$K = \text{Load} \cdot \text{Input}_0'$



Registers with Parallel Load (JK-Type Flip-Flops)

Another way of loading parallel data into a register is, to apply the data directly to the inputs of flip-flops, and apply LOAD.CLOCK signal to the clock inputs of the flip-flops.



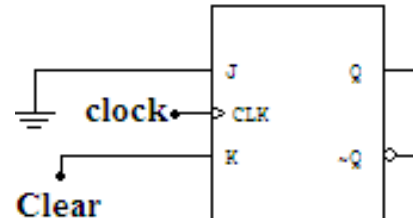
Clearing the Register

Flip-flops generally have an asynchronous Clear input which sets the output to 0. Since this input is asynchronous, it operates regardless of the clock signal. Assume that we need a synchronous Clear input. We can use the state table for a 1-bit register, given below.

Clear	Next State (Q)	J	K
0	Hold the state	0	0
1	0	0	1

$J = 0$

$K = \text{Clear}$



Incrementing the Value Stored in a 1-bit Register

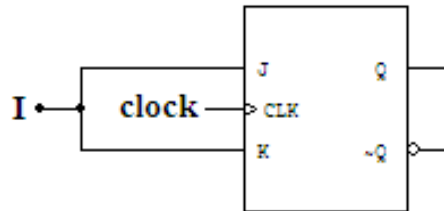
The state table for incrementing the value of a 1-bit register is given below.

Incr. (I)	Next State (Q)	J	K
0	Hold the state	0	0
1	Toggle	1	1

Incr. (I)	Present State (q)	Next State (Q)	J	K
0	0	0	0	x
0	1	1	x	0
1	0	1	1	x
1	1	0	x	1

$$J = I$$

$$K = I$$



Incrementing the Value Stored in a 2-bit Register

Incr. (I)	Present State (q1q0)	Next State (Q1Q0)	J1	K1	J0	K0
0	0 0	0 0	0	x	0	x
0	0 1	0 1	0	x	x	0
0	1 0	1 0	x	0	0	x
0	1 1	1 1	x	0	x	0
1	0 0	0 1	0	x	1	x
1	0 1	1 0	1	x	x	1
1	1 0	1 1	x	0	1	x
1	1 1	0 0	x	1	x	1

Incrementing the Value Stored in a 2-bit Register

Incr. (I)	Present State (q1q0)	Next State (Q1Q0)	J1	K1	J0	K0
0	0 0	0 0	0	x	0	x
0	0 1	0 1	0	x	x	0
0	1 0	1 0	x	0	0	x
0	1 1	1 1	x	0	x	0
1	0 0	0 1	0	x	1	x
1	0 1	1 0	1	x	x	1
1	1 0	1 1	x	0	1	x
1	1 1	0 0	x	1	x	1

q0 \ Iq1	0	1
00		
01	x	x
11	x	x
10		1

$$J1 = I.q0$$

q0 \ Iq1	0	1
00	x	x
01		
11		1
10	x	x

$$K1 = I.q0$$

q0 \ Iq1	0	1
00		x
01		x
11	1	x
10	1	x

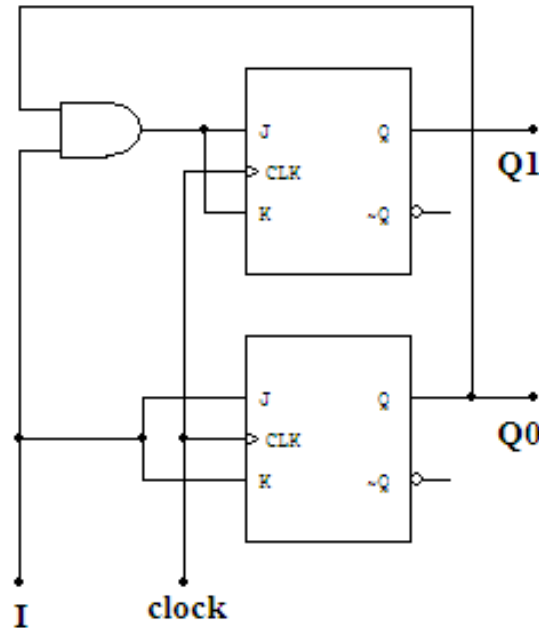
$$J0 = I$$

q0 \ Iq1	0	1
00	x	
01	x	
11	x	1
10	x	1

$$K0 = I$$

Incrementing the Value Stored in a 2-bit Register

$$\begin{aligned} J1 &= I.q0 \\ K1 &= I.q0 \\ J0 &= I \\ K0 &= I \end{aligned}$$



To increment the value in a 3-bit register, it is obvious that the inputs of the flip-flop Q2 must be $J2 = K2 = I.q1.q0$.

If we apply $I=1$ when $q1q0=11$, JK inputs of the Q2 flip-flop will be 11, and it will toggle the output. In all other circumstances, JK will be 00 and Q2 will hold its value.